

Documentación Técnica Interna

INFRAESTRUCTURA PERSONAL

GABRIEL NAVIA ALMENDRAS

Contenido

INFORME VPS-1 - WireGuard	3
Estado inicial del sistema.....	4
Instalación de WireGuard.....	4
Configuración del servidor WireGuard.....	5
Activación del reenvío IPv4.....	6
Inicio y habilitación del servicio	6
Configuración del cliente (Windows 11)	6
Configuración en Oracle Cloud.....	7
Pruebas de conectividad.....	7
Procedimiento de desmontaje y reversión completa	7
Conclusiones.....	8
[ANEXO] Sistema de Gestión Automatizada de Clientes WireGuard (Script)	9
Estructura general del script.....	9
[SCRIPT 1 - Wireguard] Bloque de Script.....	12
INFORME – VPS-2 Reverse Proxy con NGINX	18
Preparación previa antes de realizar cambios	18
Estructura de historial.....	18
Instalación de NGINX	19
UFW para permitir acceso a la VPS	19
Preparación del certificado SSL Sectigo Wildcard	19
Construcción del fullchain	19
Configuración del Reverse Proxy	20
Creación del archivo del sitio	20
Integración con WireGuard y GNS3 Server	21
Verificación final	21
Proceso de desinstalación (reversibilidad total)	21
Conclusión.....	22
[ANEXO] Autenticación con NGINX (htpasswd) y Landing Page Estática.....	23
2. Creación del archivo de credenciales	23
3. Estructura de la Landing Page	23
4. Configuración NGINX para la Landing Page	23

5. Protección del subdominio GNS3.....	24
6. Flujo de acceso final	24
Landing Page Estática (HTML).....	25
Reversión de la Configuración de Autenticación y Landing Page.....	27
INFORME – VPS-3 Instalación de Docker	29
Instalación de repositorio de docker	29
Instalación de docker engine y subcomponentes.....	30
Habilitar el servicio de docker en cuanto arranca la VPS.....	30
Reversión completa de la instalación de Docker	31
Estructura de carpetas y organización de servicios Docker	32
Estructuras de nombre y directorios	33
Redes Internas	33
Reversión de la estructura Docker	33

INFORME VPS-1 - WireGuard

Instalación, Configuración y Reversión de WireGuard en VPS Oracle Always Free (Ubuntu 24.04 ARM64)

El objetivo de este documento es describir de forma detallada y estructurada el proceso completo de instalación, configuración y desmontaje del servicio VPN WireGuard en una VPS Oracle Always Free basada en Ubuntu 24.04 ARM64.

Este tipo de instancias gratuitas tienen una característica importante: **no pueden reinstalarse ni regenerarse sin perder permanentemente el recurso**, ya que Oracle no garantiza que vuelva a estar disponible. Por ello, cualquier modificación en el sistema debe realizarse con extremo cuidado, manteniendo siempre la posibilidad de revertir los cambios y documentando el estado previo.

WireGuard se utiliza para crear una red privada virtual segura entre la VPS y otros dispositivos (PC, portátiles, servidores externos, etc.). En este caso, se ha elegido la red privada **10.10.0.0/16**, que permitirá conectar múltiples nodos en el futuro, tanto para uso personal como para proyectos académicos.

Este informe tiene tres objetivos principales:

1. **Documentar el estado inicial del sistema**, incluyendo los paquetes instalados y la configuración relevante.
 2. Describir el proceso de instalación y configuración de WireGuard, de forma segura y reversible.
 3. **Proporcionar un procedimiento completo de desmontaje**, que permita devolver la VPS a su estado original sin dejar residuos.
-

Estado inicial del sistema

Antes de instalar WireGuard, se realiza una captura del estado actual del sistema. Esto permite:

- Comparar antes/después.
- Detectar cambios no deseados.
- Facilitar la reversión en caso de error.
- Documentar el entorno para auditoría o anexos técnicos.

Exportación del listado completo de paquetes instalados

Se genera un listado exhaustivo de todos los paquetes instalados en el sistema, incluyendo dependencias automáticas:

```
dpkg --get-selections > paquetes-instalados.txt
```

Este archivo permite comparar el estado del sistema antes y después de la instalación de WireGuard.

Exportación de paquetes instalados manualmente

Para identificar únicamente los paquetes instalados explícitamente por el usuario:

```
apt-mark showmanual > paquetes-manuales.txt
```

Este archivo es útil para detectar instalaciones no deseadas o paquetes añadidos accidentalmente.

Copia del historial de APT

Se guardan los registros de instalación, actualización y eliminación de paquetes:

```
cp /var/log/apt/history.log apt-history.log
cp /var/log/apt/term.log apt-term.log
```

Estos archivos permiten auditar cualquier cambio realizado mediante APT.

Copia de la configuración de red

Netplan es crítico en Oracle Cloud. Se realiza una copia de seguridad antes de cualquier modificación:

```
cp /etc/netplan/*.yaml netplan-backup.yaml
```

Copia del estado del firewall UFW

Se registra el estado del firewall antes de aplicar reglas relacionadas con WireGuard:

```
sudo ufw status verbose > ufw-before.txt
```

Instalación de WireGuard

WireGuard está soportado de forma nativa en el kernel Oracle 6.14, por lo que no requiere módulos adicionales ni DKMS. Solo se instalan las herramientas de espacio de usuario.

Instalación de paquetes necesarios

```
sudo apt update
sudo apt install wireguard wireguard-tools
```

Paquetes instalados:

- wireguard
- wireguard-tools

Estos paquetes no modifican componentes críticos del sistema.

Generación de claves del servidor

Se generan las claves privada y pública del servidor:

```
wg genkey | tee /etc/wireguard/server.key | wg pubkey >
/etc/wireguard/server.pub
```

```
chmod 600 /etc/wireguard/server.key
```

Archivos generados:

```
/etc/wireguard/server.key
/etc/wireguard/server.pub
```

Configuración del servidor WireGuard

Archivo de configuración /etc/wireguard/wg0.conf

Se crea el archivo de configuración principal:

```
[Interface]
Address = 10.10.0.1/16
ListenPort = 51820
PrivateKey = <clave_privada_servidor>
MTU = 1420

PostUp = ufw route allow in on wg0 out on eth0
PostUp = ufw allow 51820/udp
PostDown = ufw route delete allow in on wg0 out on eth0
PostDown = ufw delete allow 51820/udp
```

Explicación de parámetros:

- Address: dirección IP del servidor dentro de la red privada WireGuard.
- ListenPort: puerto UDP utilizado para conexiones entrantes.
- PrivateKey: clave privada del servidor.
- MTU: valor recomendado para Oracle Cloud para evitar fragmentación.
- PostUp/PostDown: reglas reversibles aplicadas mediante UFW, sin modificar iptables directamente.

Activación del reenvío IPv4

WireGuard requiere que el sistema permita el reenvío de paquetes IPv4. Se habilita mediante un archivo independiente para no modificar configuraciones globales:

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-wireguard.conf
sudo sysctl --system
```

Inicio y habilitación del servicio

Se habilita la interfaz wg0 para que se active automáticamente al iniciar el sistema:

```
sudo systemctl enable wg-quick@wg0
sudo systemctl start wg-quick@wg0
Verificación del estado:
```

```
sudo wg show
```

Configuración del cliente (Windows 11)

Ejemplo de configuración para un cliente Windows:

```
[Interface]
PrivateKey = <clave_privada_cliente>
Address = 10.10.0.2/16
DNS = 1.1.1.1
MTU = 1420
```

```
[Peer]
PublicKey = <clave_publica_servidor>
AllowedIPs = 0.0.0.0/0
Endpoint = <IP_PUBLICA_VPS>:51820
PersistentKeepalive = 25
```

Relación entre claves en WireGuard

- El **servidor** almacena su clave privada y la clave pública de cada cliente.
 - Cada **cliente** almacena su clave privada y la clave pública del servidor.
 - Las claves privadas nunca se comparten.
 - Las claves públicas se intercambian entre pares.
 - Por ello, en los archivos de configuración de los clientes siempre aparece la clave pública del servidor, pero nunca su clave privada.
-

Configuración en Oracle Cloud

Es necesario permitir el tráfico entrante en el puerto UDP 51820 desde la consola de Oracle Cloud. En la Security List o Network Security Group asociado a la VNIC:

- Protocolo: UDP
- Puerto: 51820
- Origen: 0.0.0.0/0

Este cambio no afecta a la VPS, solo a la red virtual de Oracle.

Pruebas de conectividad

Desde el cliente:

```
ping 10.10.0.1
```

Desde la VPS:

```
ping 10.10.0.2
```

Logs del servicio:

```
sudo journalctl -u wg-quick@wg0 -n 50
```

Procedimiento de desmontaje y reversión completa

El desmontaje debe devolver la VPS a su estado original, sin dejar configuraciones residuales, reglas de firewall, claves o archivos de sistema.

A continuación, se detalla el proceso completo, con explicación de cada paso.

Detener y deshabilitar el servicio

```
sudo systemctl stop wg-quick@wg0  
sudo systemctl disable wg-quick@wg0
```

Esto garantiza que WireGuard no se iniciará automáticamente.

Eliminar reglas de firewall aplicadas

```
sudo ufw delete allow 51820/udp  
sudo ufw route delete allow in on wg0 out on eth0
```

Estas reglas fueron añadidas por PostUp, por lo que deben eliminarse manualmente.

10.3 Eliminar archivos de configuración y claves

```
sudo rm /etc/wireguard/wg0.conf  
sudo rm /etc/wireguard/server.key  
sudo rm /etc/wireguard/server.pub
```

Esto elimina cualquier rastro de configuración o credenciales.

Eliminar configuración de reenvío IP

```
sudo rm /etc/sysctl.d/99-wireguard.conf  
sudo sysctl -system
```

Esto restaura el comportamiento original del sistema.

Desinstalar paquetes (opcional)

```
sudo apt remove wireguard wireguard-tools
```

Solo si se desea eliminar completamente el software.

Verificación final

Comparar:

- paquetes-instalados.txt
- paquetes-manuales.txt
- ufw-before.txt

Esto permite confirmar que el sistema ha vuelto a su estado original.

Conclusiones

El procedimiento descrito permite instalar, configurar y desmontar WireGuard en una VPS de forma segura, controlada y totalmente reversible. No se modifican componentes críticos del sistema, no se altera la configuración de red base y todas las reglas aplicadas son reversibles.

[ANEXO] Sistema de Gestión Automatizada de Clientes WireGuard (Script)

Con el fin de facilitar la administración de clientes en el servidor WireGuard, se ha desarrollado un script monolítico denominado `wireguardconf.sh`. Este script centraliza todas las operaciones necesarias para gestionar la incorporación, eliminación, listado y generación de clientes, incluyendo la creación de archivos de configuración y códigos QR.

Este módulo constituye una herramienta de administración avanzada que permite mantener un control preciso sobre los peers registrados en el servidor, evitando errores manuales y garantizando la coherencia de la configuración.

Objetivos del script

El script `wireguardconf.sh` se ha diseñado para cumplir los siguientes objetivos:

1. Automatizar la gestión de clientes WireGuard.
2. Evitar la edición manual del archivo `wg0.conf`.
3. Garantizar la integridad de la configuración del servidor.
4. Registrar todas las acciones en un archivo de log.
5. Validar direcciones IP para evitar conflictos.
6. Generar archivos `.conf` completos para clientes.
7. Permitir la creación opcional de códigos QR para dispositivos móviles.
8. Ofrecer un menú interactivo para facilitar su uso.

Estructura general del script

El script implementa las siguientes funciones principales:

Registro de clientes (-r)

Permite añadir un nuevo peer al servidor proporcionando:

- Clave pública del cliente
- Dirección IP asignada
- Nombre identificativo

El script valida la IP, añade el bloque `[Peer]` al final de `wg0.conf` y recarga la configuración sin interrumpir conexiones activas.

Baja de clientes (-b)

Elimina un cliente existente identificándolo por su nombre. El script localiza el comentario `# Nombre Cliente` y elimina todo el bloque asociado.

Listado de clientes (-l)

Muestra todos los clientes registrados, indicando:

- Nombre
- Dirección IP asignada

Esto permite auditar rápidamente el estado de la VPN.

Generación completa de clientes (-g)

Genera:

- Par de claves del cliente
- Archivo .conf completo
- Registro automático en el servidor
- Código QR opcional

Los archivos generados se almacenan en ~/gen.

Menú interactivo (-m)

Proporciona una interfaz textual que permite:

- Registrar clientes
- Dar de baja
- Listar
- Generar configuraciones
- Salir

Este menú facilita el uso del sistema sin necesidad de recordar parámetros.

Validación de direcciones IP

El script implementa un sistema de validación que garantiza:

1. Que la IP pertenece a la red 10.10.0.0/16.
2. Que la IP no está ya asignada a otro cliente.
3. Que el formato es correcto.

Esto evita conflictos de direccionamiento dentro de la VPN.

Sistema de logs

Todas las acciones realizadas por el script se registran en:

Código

/var/log/wireguard-admin.log

Cada entrada incluye:

- Fecha y hora
- Acción realizada
- Nombre del cliente
- Dirección IP

Este registro permite auditar cambios y mantener trazabilidad.

Directorio de generación

Los archivos de configuración generados se almacenan en:

Código

~/gen

Este directorio contiene:

- Archivos .conf de clientes
- Códigos QR opcionales
- Claves privadas y públicas generadas

Esto permite mantener una estructura ordenada y separada de la configuración del servidor.

Funcionamiento interno

El script utiliza las siguientes técnicas:

1. **Manipulación segura de archivos** mediante awk y archivos temporales.
2. **Recarga no disruptiva** de WireGuard mediante:

Código

```
wg synccnf wg0 <(wg-quick strip wg0)
```

Esto evita reinicios de la interfaz y mantiene las conexiones activas.

1. **Generación de claves** con wg genkey y wg pubkey.
2. Creación de códigos QR mediante qrencode.
3. Uso de comentarios estructurados para identificar clientes.

Flujo de trabajo recomendado

- Generación de un cliente:

Código:

```
./wireguardconf.sh -g NombreCliente 10.10.0.X
```

- Exportación del archivo .conf al dispositivo correspondiente.
- Registro automático en el servidor.
- Baja del cliente cuando sea necesario:

Código : .

```
/wireguardconf.sh -b NombreCliente
```

- Auditoría:

Código

```
./wireguardconf.sh -l
```

[SCRIPT 1 - Wireguard] Bloque de Script

```
#!/bin/bash

WGCONF="/etc/wireguard/wg0.conf"
GENDIR="/home/ubuntu/scripts/wireguard-gen"
LOGFILE="/var/log/wireguard-admin.log"

# =====
#  PARÁMETROS CONFIGURABLES
#  =====
WG_CIDR="10.10.0.0/16"
WG_NET="10.10.0"
WG_MASK="16"

WG_SERVER_PRIV="/etc/wireguard/server.key"
WG_SERVER_PUB="/etc/wireguard/server.pub"

# Colores
GREEN="\e[32m"
RED="\e[31m"
YELLOW="\e[33m"
BLUE="\e[34m"
NC="\e[0m"

mkdir -p "$GENDIR"

log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" >> "$LOGFILE"
}

# =====
#  PROTECCIÓN DEL ARCHIVO BASE
#  =====
proteger_archivo_base() {
    if [ ! -s "$WGCONF" ]; then
        echo -e "${YELLOW}[WARN] wg0.conf está vacío o no existe. Se
reconstruirá.${NC}"
        {
            echo "[Interface]"
            echo "Address = ${WG_NET}.1/${WG_MASK}"
            echo "ListenPort = 51820"
            echo "PrivateKey = $(cat $WG_SERVER_PRIV)"
            echo "MTU = 1420"
            echo "PostUp = ufw route allow in on wg0 out on eth0"
            echo "PostUp = ufw allow 51820/udp"
            echo "PostDown = ufw route delete allow in on wg0 out on eth0"
            echo "PostDown = ufw delete allow 51820/udp"
            echo ""
        } > "$WGCONF"

        echo -e "${GREEN}[OK] Archivo base reconstruido.${NC}"
        log "Archivo base wg0.conf reconstruido automáticamente"
    fi
}
```

```
# =====
#  BACKUP AUTOMÁTICO
#  =====
backup() {
    TS=$(date '+%Y%m%d-%H%M%S')
    cp "$WGCONF" "/etc/wireguard/wg0.conf.bak-$TS"
    echo -e "${BLUE}[INFO] Backup creado: /etc/wireguard/wg0.conf.bak-$TS${NC}"
    log "Backup creado: wg0.conf.bak-$TS"
}

# =====
#  VALIDACIÓN DE IP (CIDR REAL)
#  =====
validar_ip() {
    IP=$1

    # Extraer red y máscara del CIDR
    NET=$(echo "$WG_CIDR" | cut -d'/' -f1)
    MASK=$(echo "$WG_CIDR" | cut -d'/' -f2)

    # Convertir IP y red a enteros
    ip_int=$(printf "%u\n" $(echo "$IP" | awk -F. '{print ($1*256^3)+($2*256^2)+($3*256)+$4}'))
    net_int=$(printf "%u\n" $(echo "$NET" | awk -F. '{print ($1*256^3)+($2*256^2)+($3*256)+$4}'))

    # Máscara en entero
    mask_int=$(( 0xFFFFFFFF << (32 - MASK) & 0xFFFFFFFF ))

    # Validar pertenencia al CIDR
    if ! (( (ip_int & mask_int) == (net_int & mask_int) )); then
        echo -e "${RED}[ERROR] La IP $IP no pertenece a ${WG_CIDR}${NC}"
        exit 1
    fi

    # Validar que no esté ya asignada
    if grep -q "$IP/32" "$WGCONF"; then
        echo -e "${RED}[ERROR] La IP $IP ya está asignada.${NC}"
        exit 1
    fi
}

# =====
#  REGISTRAR CLIENTE
#  =====
registrar() {
    proteger_archivo_base
    backup

    PUBKEY=$1
    IP=$2
    NAME=$3
```

```

validar_ip "$IP"

{
    echo ""
    echo "# $NAME"
    echo "[Peer]"
    echo "PublicKey = $PUBKEY"
    echo "AllowedIPs = $IP/32"
} >> "$WGCONF"

wg synccnf wg0 <(wg-quick strip wg0)

echo -e "${GREEN}[OK] Cliente '$NAME' registrado con IP $IP${NC}"
log "Registrado cliente $NAME con IP $IP"
}

# =====
# BAJA DE CLIENTE (CORREGIDA)
# =====
baja() {
    proteger_archivo_base
    backup

    NAME=$1
    TMP=$(mktemp)

    if ! grep -q "^# $NAME$" "$WGCONF"; then
        echo -e "${RED}[ERROR] No existe un cliente con nombre
'$NAME'${NC}"
        exit 1
    fi

    awk -v name="$NAME" '
    BEGIN {del=0}
    {
        if ($0 == "# "name) {del=1; next}

        if (del && index($0, "[Peer]") == 1) {next}
        if (del && index($0, "PublicKey") == 1) {next}
        if (del && index($0, "AllowedIPs") == 1) {del=0; next}

        if (!del) print $0
    }
    ' "$WGCONF" > "$TMP"

    mv "$TMP" "$WGCONF"
    wg synccnf wg0 <(wg-quick strip wg0)

    echo -e "${GREEN}[OK] Cliente '$NAME' eliminado${NC}"
    log "Eliminado cliente $NAME"
}

# =====
# LISTAR CLIENTES
# =====
listar() {

```

```
proteger_archivo_base

echo -e "${BLUE}Clientes registrados:${NC}"
awk '
    /^#/ {name=$2}
    /AllowedIPs/ {ip=$3; print " -", name, ip}
' "$WGCONF"

log "Listado de clientes mostrado"
}

# =====
# GENERAR CLIENTE COMPLETO
# =====
generar() {
    proteger_archivo_base
    backup

    NAME=$1
    IP=$2
    QR=$3

    validar_ip "$IP"

    PRIV=$(wg genkey)
    PUB=$(echo "$PRIV" | wg pubkey)
    SERVERPUB=$(cat $WG_SERVER_PUB)

    CONF="$GENDIR/$NAME.conf"

    cat > "$CONF" <<EOF
[Interface]
PrivateKey = $PRIV
Address = $IP/$WG_MASK
DNS = 1.1.1.1
MTU = 1420

[Peer]
PublicKey = $SERVERPUB
AllowedIPs = $WG_CIDR
Endpoint = oci.gabrielnav.es:51820
PersistentKeepalive = 25
EOF

    chown ubuntu:ubuntu "$CONF"

    registrar "$PUB" "$IP" "$NAME"

    echo -e "${GREEN}[OK] Archivo generado: $CONF${NC}"

    if [ "$QR" == "yes" ]; then
        qrencode -o "$GENDIR/$NAME.png" < "$CONF"
        chown ubuntu:ubuntu "$GENDIR/$NAME.png"
        echo -e "${GREEN}[OK] QR generado: $GENDIR/$NAME.png${NC}"
    fi
}
```



```
    log "Generado cliente $NAME con IP $IP"
}

# =====
#  MENÚ
# =====
menu() {
    while true; do
        echo ""
        echo -e "${BLUE}===== Administrador WireGuard =====${NC}"
        echo "1) Registrar cliente"
        echo "2) Dar de baja cliente"
        echo "3) Listar clientes"
        echo "4) Generar cliente completo"
        echo "5) Salir"
        echo -n "Seleccione una opción: "
        read opt

        case $opt in
            1)
                echo -n "Clave pública: "; read PUB
                echo -n "IP: "; read IP
                echo -n "Nombre: "; read NAME
                registrar "$PUB" "$IP" "$NAME"
                ;;
            2)
                echo -n "Nombre: "; read NAME
                baja "$NAME"
                ;;
            3)
                listar
                ;;
            4)
                echo -n "Nombre: "; read NAME
                echo -n "IP: "; read IP
                echo -n "Generar QR (y/n): "; read q
                [ "$q" == "y" ] && QR="yes" || QR="no"
                generar "$NAME" "$IP" "$QR"
                ;;
            5)
                exit 0
                ;;
        esac
    done
}

# =====
#  PARÁMETROS
# =====
case $1 in
    -r) registrar $2 $3 $4 ;;
    -b) baja $2 ;;
    -l) listar ;;
    -g)
```

```
        if [ "$4" == "-q" ]; then
            generar $2 $3 yes
        else
            generar $2 $3 no
        fi
        ;;
-m) menu ;;
*)
    echo "Uso:"
    echo "  -r PUBKEY IP Nombre      Registrar cliente"
    echo "  -b Nombre                Dar de baja cliente"
    echo "  -l                       Listar clientes"
    echo "  -g Nombre IP [-q]       Generar cliente y registrar"
    echo "  -m                       Menú interactivo"
    ;;
esac
```

INFORME – VPS-2 Reverse Proxy con NGINX

El propósito de este módulo es permitir que el servicio **GNS3 Web Server**, alojado en un equipo Windows 11 dentro de la red privada WireGuard (IP 10.10.0.2, puerto 3080), pueda ser accedido de forma segura desde Internet mediante el dominio:

`https://gns.gabrielnav.es`

Para ello, se despliega un **reverse proxy NGINX** en el VPS, encargado de:

1. Recibir conexiones HTTPS desde Internet.
2. Validar el certificado SSL Sectigo Wildcard (*.gabrielnav.es).
3. Detectar el subdominio solicitado (gns.gabrielnav.es).
4. **Reenviar la petición** hacia el servicio interno GNS3 Server a través de WireGuard.
5. **Mantener compatibilidad con WebSockets**, necesarios para la interfaz de GNS3.

Este diseño garantiza:

- Seguridad mediante TLS.
- Aislamiento del servicio interno.
- Reversibilidad total.
- Independencia del equipo Windows respecto a Internet.
- Integración limpia con la red privada WireGuard.

Preparación previa antes de realizar cambios

Antes de instalar o modificar cualquier componente, se realizó una copia del estado del sistema para garantizar la reversibilidad.

Estructura de historial

Se creó la carpeta:

`historial/VPS-2-NGINX-REVERSE-PROXY/`

Esta carpeta almacena:

- Listados de paquetes antes y después de la instalación.
- Copias de seguridad de configuraciones.
- Registros del firewall.
- Archivos de configuración del reverse proxy.

Copia del listado de paquetes instalados

```
dpkg -l > historial/VPS-2-NGINX-REVERSE-PROXY/paquetes-instalados.txt
```

Esto permite restaurar el estado original del VPS si fuera necesario.

Copia del estado del firewall

```
sudo ufw status verbose > historial/VPS-2-NGINX-REVERSE-PROXY/ufw-before.txt
```

Instalación de NGINX

NGINX se instala mediante:

```
sudo apt update
sudo apt install nginx
```

Tras la instalación, se registra el nuevo estado de paquetes:

```
dpkg -l > historial/VPS-2-NGINX-REVERSE-PROXY/paquetes-post-
instalacion.txt
```

UFW para permitir acceso a la VPS

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw reload
```

Preparación del certificado SSL Sectigo Wildcard

El administrador dispone de los siguientes archivos proporcionados por IONOS/Sectigo:

- gabrielnave.es_ssl_certificate.cer → Certificado principal
- intermediate1.cer → Certificado intermedio
- intermediate2.cer → Segundo intermedio
- SSL.gabrienave.es.key → Clave privada

Creación de carpeta segura

```
sudo mkdir -p /etc/nginx/ssl/gabrielnave.es
sudo chmod 700 /etc/nginx/ssl/gabrielnave.es
```

Subida de archivos

Los archivos se copiarán a la carpeta anterior mediante WinSCP o scp.

Construcción del fullchain

NGINX requiere un archivo que contenga:

- Certificado principal
- Certificados intermedios

Se genera así:

```
cd /etc/nginx/ssl/gabrielnave.es
sudo sh -c 'cat gabrielnave.es_ssl_certificate.cer intermediate1.cer
intermediate2.cer > fullchain.crt'
```

Permisos estrictos

```
sudo chmod 600 /etc/nginx/ssl/gabrielnave.es/*
sudo chown root:root /etc/nginx/ssl/gabrielnave.es/*
```

Esto evita accesos no autorizados.

Configuración del Reverse Proxy

El reverse proxy se configura para:

- Redirigir HTTP → HTTPS
- Servir HTTPS con el certificado Sectigo
- Reenviar tráfico hacia 10.10.10.10:80
- Mantener WebSockets

Creación del archivo del sitio

```
sudo nano /etc/nginx/sites-available/gns.gabrielnav.es.conf
```

[NGINX]

```
server {
    listen 80;
    server_name gns.gabrielnav.es;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name gns.gabrielnav.es;

    ssl_certificate      /etc/nginx/ssl/gabrielnav.es/fullchain.crt;
    ssl_certificate_key  /etc/nginx/ssl/gabrielnav.es/SSL.gabrienav.es.key;

    location / {
        proxy_pass http://10.10.10.10:80;

        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_read_timeout 3600;
        proxy_send_timeout 3600;

        proxy_buffering off;
    }
}
```

Activación del sitio

```
sudo ln -s /etc/nginx/sites-available/gns.gabrielnav.es.conf \
    /etc/nginx/sites-enabled/
```

Validación y recarga

```
sudo nginx -t
sudo systemctl reload nginx
```

Integración con WireGuard y GNS3 Server

Para que el reverse proxy funcione, se cumplieron los siguientes requisitos:

GNS3 VM

- Configurado para Bind to: 10.10.10.10
- Escuchando en el puerto **80**
- Firewall de la VM permite tráfico desde **10.10.0.0/16** hacia el puerto 80

Prueba desde el VPS

```
curl http://10.10.10.10:80
```

La respuesta JSON confirmaría que el servicio es accesible a través de WireGuard.

Verificación final

Desde navegador externo

<https://gns.gabrielnav.es>

Debe mostrar:

- Certificado Sectigo válido
- Interfaz de GNS3
- WebSockets funcionando

Desde el VPS

```
curl -I https://gns.gabrielnav.es
```

Debe devolver:

```
HTTP/2 200  
server: nginx
```

Proceso de desinstalación (reversibilidad total)

Si en el futuro se desea revertir completamente este módulo, se deben seguir estos pasos:

Desactivar el sitio

```
sudo rm /etc/nginx/sites-enabled/gns.gabrielnav.es.conf
```

Recargar NGINX

```
sudo systemctl reload nginx
```

Eliminar archivos SSL

```
sudo rm -r /etc/nginx/ssl/gabrielnav.es
```

Desinstalar NGINX

```
sudo apt remove nginx nginx-common  
sudo apt autoremove
```

Consultar historial

Todos los archivos necesarios para restaurar el estado previo se encuentran en:

`historial/VPS-2-NGINX-REVERSE-PROXY/`

Resultado final

El VPS actúa como un reverse proxy seguro y eficiente:

Código

Cliente → HTTPS (Sectigo) → VPS (NGINX) → WireGuard → Windows 11 → GNS3 Server

Conclusión

Este módulo de infraestructura deja documentado un proceso completo, seguro y totalmente reversible para exponer un servicio interno —GNS3 Server en Windows 11— a través de un dominio público con cifrado TLS y control total del flujo de tráfico. La solución se apoya en tres pilares fundamentales:

- **Aislamiento:** el servicio real nunca se expone directamente a Internet; solo es accesible a través de la red privada WireGuard.
- **Seguridad:** el uso del certificado Sectigo Wildcard garantiza comunicaciones cifradas y verificadas, mientras que los permisos estrictos y la custodia segura de claves privadas reducen riesgos operativos.
- **Reversibilidad:** cada paso queda registrado en el historial del VPS, permitiendo deshacer cualquier cambio incluso meses o años después, sin necesidad de recordar detalles técnicos.

El reverse proxy NGINX actúa como punto de entrada controlado, validando el dominio, gestionando el cifrado y reenviando el tráfico hacia el servicio interno mediante reglas claras y mantenibles. La arquitectura resultante es robusta, modular y preparada para crecer, permitiendo añadir nuevos subdominios, servicios o certificados sin alterar la base del sistema.

[ANEXO] Autenticación con NGINX (htpasswd) y Landing Page Estática

Implementar un sistema de autenticación simple y seguro para proteger:

- La landing page privada.
- El acceso directo al servicio GNS3 Web UI.
- Cualquier otro servicio interno expuesto mediante NGINX.

La autenticación se realiza mediante `auth_basic` y un archivo `.htpasswd`, sin necesidad de PHP, bases de datos o contenedores.

2. Creación del archivo de credenciales

Crear el archivo `.htpasswd` con el primer usuario: (necesario instalar `apache2-utils`)

```
sudo htpasswd -c /etc/nginx/.htpasswd gabriel
```

Añadir más usuarios:

```
sudo htpasswd /etc/nginx/.htpasswd invitado
```

El archivo queda almacenado en:

```
/etc/nginx/.htpasswd
```

3. Estructura de la Landing Page

Ubicación recomendada:

```
/var/www/landing/
```

4. Configuración NGINX para la Landing Page

Archivo de configuración:

```
server {
    listen 80;
    server_name portal.gabrielnav.es www.gabrielnav.es;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl http2;
    server_name portal.gabrielnav.es www.gabrielnav.es;

    ssl_certificate      /etc/nginx/ssl/gabrielnav.es/fullchain.crt;
    ssl_certificate_key  /etc/nginx/ssl/gabrielnav.es/SSL.gabrienav.es.key;

    auth_basic "Portal de Acceso - VPS";
    auth_basic_user_file /etc/nginx/.htpasswd;

    root /var/www/landing;
    index index.html;
}
```

Esto asegura que cualquier acceso a la landing page requiera autenticación.

5. Protección del subdominio GNS3

```
server {
    listen 443 ssl http2;
    server_name gns.gabrielnav.es;

    ssl_certificate      /etc/nginx/ssl/fullchain.crt;
    ssl_certificate_key  /etc/nginx/ssl/privkey.key;

    auth_basic "GNS3 Access";
    auth_basic_user_file /etc/nginx/.htpasswd;

    location / {
        proxy_pass http://10.10.10.10:80;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_read_timeout 3600;
        proxy_send_timeout 3600;
        proxy_buffering off;
    }
}
```

Resultado:

- Si alguien accede directamente a gns.gabrielnav.es, se solicita login.
- Si accede desde la landing (ya autenticado), entra directamente.
- GNS3 queda completamente protegido.

6. Flujo de acceso final

www.gabrielnav.es → Login → Landing privada → Acceso a GNS3
gns.gabrielnav.es → Login directo si no hay sesión previa

Landing Page Estática (HTML)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Gabriel Lab Portal</title>
  <style>
    body {
      margin: 0;
      background: #0d0d0d;
      font-family: Arial, sans-serif;
      color: #e6e6e6;
      display: flex;
      flex-direction: column;
      align-items: center;
      padding: 40px;
    }
    h1 {
      font-size: 2.4rem;
      margin-bottom: 10px;
      color: #4da6ff;
    }
    p {
      color: #bfbfbf;
      margin-bottom: 40px;
    }
    .grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
      gap: 20px;
      width: 80%;
      max-width: 900px;
    }
    .card {
      background: #1a1a1a;
      padding: 25px;
      border-radius: 10px;
      text-align: center;
      border: 1px solid #333;
      transition: 0.2s;
    }
    .card:hover {
      border-color: #4da6ff;
      transform: translateY(-3px);
    }
    a {
      text-decoration: none;
      color: #4da6ff;
      font-weight: bold;
    }
  </style>
</head>
```

```
<body>

  <h1>Gabriel Lab Portal</h1>
  <p>Acceso privado a servicios internos del laboratorio.</p>

  <div class="grid">
    <div class="card">
      <h2>GNS3 Web UI</h2>
      <p>Simulador de redes</p>
      <a href="https://gns.gabrielnav.es">Acceder</a>
    </div>

    <div class="card">
      <h2>Nextcloud</h2>
      <p>Almacenamiento privado</p>
      <a href="#">Próximamente</a>
    </div>

    <div class="card">
      <h2>VM PXE</h2>
      <p>Infraestructura de despliegue</p>
      <a href="#">Próximamente</a>
    </div>

    <div class="card">
      <h2>Más servicios</h2>
      <p>En desarrollo</p>
      <a href="#">Ver</a>
    </div>
  </div>

</body>
</html>
```

Reversión de la Configuración de Autenticación y Landing Page

Objetivo

Este anexo documenta el procedimiento para **revertir completamente** la configuración aplicada para:

- Autenticación mediante auth_basic y .htpasswd.
- Landing page estática servida por NGINX.
- Protección del subdominio gns.gabrielnav.es.

El objetivo es restaurar el entorno a un estado **minimalista**, sin autenticación y sin landing page, manteniendo únicamente el reverse proxy hacia GNS3.

Eliminación de la autenticación (htpasswd)

Para revertir la autenticación, basta con eliminar o comentar las siguientes líneas en los bloques server {} afectados:

```
auth_basic "Gabriel Portal";  
auth_basic_user_file /etc/nginx/.htpasswd;
```

y en el caso del subdominio GNS3:

```
auth_basic "GNS3 Access";  
auth_basic_user_file /etc/nginx/.htpasswd;
```

Tras eliminarlas, NGINX dejará de solicitar credenciales.

Eliminación del archivo de credenciales

Si se desea eliminar completamente el archivo .htpasswd:

```
sudo rm /etc/nginx/.htpasswd
```

Esto elimina todos los usuarios y desactiva cualquier autenticación basada en este archivo.

Eliminación de la landing page

Si se desea revertir la landing page:

1. Eliminar el directorio:

```
sudo rm -r /var/www/landing
```

2. Eliminar del bloque server correspondiente las líneas:

```
root /var/www/landing;  
index index.html;
```

3. Si la landing ya no se utilizará, puede eliminarse el bloque server completo asociado a www.gabrielnav.es.

Restauración del reverse proxy de GNS3

Para dejar el subdominio gns.gabrielnav.es en su estado original, sin autenticación ni landing, el bloque debe quedar así:

```
server {
    listen 443 ssl http2;
    server_name gns.gabrielnav.es;

    ssl_certificate      /etc/nginx/ssl/fullchain.crt;
    ssl_certificate_key  /etc/nginx/ssl/privkey.key;

    location / {
        proxy_pass http://10.10.10.10:80;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_read_timeout 3600;
        proxy_send_timeout 3600;
        proxy_buffering off;
    }
}
```

Este bloque deja el acceso a GNS3 exactamente igual que antes de aplicar autenticación o landing page.

Recarga de NGINX

Tras realizar cualquier cambio, validar y recargar:

```
sudo nginx -t
sudo systemctl reload nginx
```

Esto aplica la reversión sin necesidad de reiniciar servicios adicionales.

Resultado final

Tras completar estos pasos:

- La autenticación desaparece por completo.
- La landing page deja de existir o de estar servida.
- El archivo .htpasswd queda eliminado (opcional).
- El subdominio GNS3 vuelve a ser accesible sin login.
- El VPS recupera su estado minimalista original.

INFORME – VPS-3 Instalación de Docker

Docker no se instala desde los repositorios estándar de Ubuntu, ya que estos suelen contener versiones antiguas y con menor soporte. En una infraestructura donde la estabilidad y la reversibilidad son fundamentales, es preferible utilizar el repositorio oficial de Docker, que garantiza actualizaciones constantes, compatibilidad y un ciclo de vida predecible.

Antes de instalar nada, se eliminan posibles restos de instalaciones previas. Esto evita conflictos de dependencias y asegura que el sistema parte de un estado limpio:

```
sudo apt remove docker docker-engine docker.io containerd runc
```

Instalación de repositorio de docker

A continuación, se instalan las dependencias necesarias para añadir repositorios HTTPS de forma segura. Estas herramientas permiten validar firmas, gestionar certificados y descargar claves GPG:

```
sudo apt update
sudo apt install ca-certificates curl gnupg
```

Se crea el directorio donde se almacenará la clave GPG del repositorio de Docker. Mantener las claves en `/etc/apt/keyrings/` es una buena práctica moderna, ya que evita mezclar claves con otros componentes del sistema:

```
sudo install -m 0755 -d /etc/apt/keyrings
```

Se descarga e importa la clave oficial de Docker, y se ajustan los permisos para que APT pueda leerla sin problemas:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Una vez validada la clave, se añade el repositorio oficial de Docker para la versión de Ubuntu instalada en la VPS. Esto permite instalar Docker directamente desde su fuente oficial:

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Instalación de docker engine y subcomponentes

Se actualiza el índice de paquetes y se instala Docker Engine junto con sus componentes principales. Esta combinación proporciona el motor de contenedores, la CLI, el runtime y el soporte para docker compose:

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

Para verificar que Docker funciona correctamente, se ejecuta el contenedor de prueba oficial:

```
sudo docker run hello-world
```

Si la salida indica que el contenedor se ha ejecutado correctamente, Docker está operativo. Por habilita el servicio para que Docker se inicie automáticamente con el sistema:

Habilitar el servicio de docker en cuanto arranca la VPS

```
sudo systemctl enable docker
sudo systemctl start docker
```

Opcionalmente, se puede añadir el usuario ubuntu al grupo docker para permitir ejecutar comandos sin sudo. Esta decisión tiene implicaciones de seguridad, ya que el grupo docker otorga privilegios equivalentes a root:

```
sudo usermod -aG docker ubuntu
```

Finalmente, se registra el estado del sistema tras la instalación, permitiendo comparar con el estado inicial y facilitar auditorías o reversión:

```
dpkg -l > historial/VPS-3-DOCKER/paquetes-post-instalacion.txt
sudo ufw status verbose > historial/VPS-3-DOCKER/ufw-after.txt
```

Reversión completa de la instalación de Docker

Si se desea retirar Docker del sistema y devolver la VPS a un estado lo más cercano posible al original, se sigue un procedimiento ordenado.

Primero se detiene el servicio:

```
sudo systemctl stop Docker
```

Se desinstalan los paquetes relacionados con Docker y sus componentes:

```
sudo apt remove docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin  
sudo apt autoremove
```

Se eliminan el repositorio y la clave GPG asociada:

```
sudo rm /etc/apt/sources.list.d/docker.list  
sudo rm /etc/apt/keyrings/docker.gpg
```

Se borran los directorios de datos de Docker y containerd, eliminando imágenes, contenedores y capas:

```
sudo rm -rf /var/lib/docker  
sudo rm -rf /var/lib/containerd
```

Si durante el uso de Docker se modificaron reglas de firewall, se puede restaurar UFW a un estado limpio:

```
sudo ufw reset  
sudo ufw enable
```

Por último, se registra el estado final:

```
dpkg -l > historial/VPS-3-DOCKER/paquetes-final.txt
```

Con esto, Docker queda completamente eliminado del sistema.

Estructura de carpetas y organización de servicios Docker

Antes de desplegar servicios en contenedores, es fundamental definir una estructura clara y coherente en el sistema de archivos. Esto permite mantener la infraestructura ordenada, separar responsabilidades y facilitar tanto el mantenimiento como la reversión.

Se centraliza todo lo relacionado con Docker en `/srv/docker/`, una ruta estándar para servicios en servidores Linux. Esto evita mezclar configuraciones con el sistema base y facilita la gestión.

Se crea la estructura base:

```
sudo mkdir -p /srv/docker
sudo mkdir -p /srv/docker/compose
sudo mkdir -p /srv/docker/public
sudo mkdir -p /srv/docker/internal
sudo mkdir -p /srv/docker/volumes
```

Cada carpeta tiene un propósito específico:

- **compose/** → archivos docker-compose.yml organizados por servicio
- **public/** → datos asociados a servicios expuestos públicamente
- **internal/** → servicios accesibles solo desde WireGuard
- **volumes/** → almacenamiento persistente de datos

Una estructura más detallada por servicio podría ser:

```
/srv/docker/
  compose/
    nextcloud/
    guacamole/
    web/
    tools/
  public/
    nextcloud/
    web/
  internal/
    guacamole/
    admin-tools/
  volumes/
    nextcloud/
    guacamole/
    db/
```

Esta organización permite eliminar un servicio completo sin afectar a los demás, simplemente retirando su directorio y su configuración asociada.

Para reforzar la seguridad, se ajustan permisos y propiedad del árbol `/srv/docker/`:

```
sudo chown -R root:root /srv/docker
sudo chmod -R 750 /srv/Docker
```

Estructuras de nombre y directorios

Además, se definen estándares de nombres para contenedores, redes y volúmenes, facilitando la administración:

- contenedores públicos → pub-*
- contenedores internos → int-*
- redes → net-*
- volúmenes → vol-*

Ejemplos:

```
pub-nextcloud
int-guacamole
net-public
net-internal
vol-nextcloud-db
vol-guacamole-config
```

Redes Internas

Dos redes Docker principales se definen, alineadas con la separación entre servicios públicos e internos:

Red para servicios públicos – Comunicación directa con NGINX-Proxy:

```
docker network create net-public
```

Red para servicios internos – Solo accesible desde la red Wireguard o Interna:

```
docker network create net-internal
```

Finalmente, se registra la estructura creada:

```
tree /srv/docker > historial/VPS-3-DOCKER/estructura-inicial.txt
```

Reversión de la estructura Docker

Si se desea deshacer la preparación de la infraestructura Docker, se eliminan las redes definidas:

```
docker network rm net-public
docker network rm net-internal
```

Y se elimina la estructura creada:

```
sudo rm -rf /srv/docker
```

Se puede registrar el estado final:

```
tree / > historial/VPS-3-DOCKER/estructura-final.txt
```

Con esto, la preparación de la infraestructura Docker queda completamente revertida.